Database & Web Application for Eukonkanto

Wife Carrying World Championships (Sonkajärvi, Finland)

Section 4.3 Functional Requirements & SQL Queries Specification

Team 20 (Team Tangled) | **Course: Data and Applications** | **Project Phase 4**

Paradigm: Relational | *DBMS: MariaDB 11.8 / MySQL 8.0 (InnoDB)* | *Stack: Python 3, Flask, PyMySQL*

Specification 4.3 Requirement Compliance

According to **Section 4.3 (Functional Requirements Implementation)**:

> *"You must implement at least **five distinct query (read) operations** and **three distinct update (write) operations**. The write operations must include at least one create, one update and one delete."*

All operations are implemented in the Python/Flask web portal using raw, parameterized SQL queries via PyMySQL and executed against the normalized 3NF schema. Multi-statement writes are protected by atomic ACID transactions.

1 Summary of Implemented Features & SQL Operations

#	Req. Type	SQL Command	Feature Name	URL / Route	Key Database Technique
R1	Read 1	`SELECT`	Approved Entries	`/entries`	6-table `JOIN`, `CONCAT_WS`, parameterized filtering
R2	Read 2	`SELECT`	Competitor Search	`/search`	Wildcard `LIKE %s` across concatenated names
R3	Read 3	`SELECT`	Category Breakdown	`/categories`	`COUNT`, conditional `SUM(CASE WHEN ...)`
R4	Read 4	`SELECT`	Heat Result Sheet	`/results`	Window function `MIN(...) OVER ()`, delta calc
R5	Read 5	`SELECT`	Category Avg Times	`/categories#avg`	Statistical aggregation (`AVG`, `MIN`, `MAX`)
W1	Write 1 (**Create**)	`INSERT`	Register a Pair	`/register`	Public (default `Pending`) & Admin pair registration
W2	Write 2 (**Update**)	`UPDATE`	Update Country	`/participants/..`	Admin-only parameterized `UPDATE` with FK validation
W3	Write 3 (**Delete**)	`DELETE`	Delete / Approve	`/entries/manage`	Admin-only multi-table `DELETE`, orphan cleanup, approval

Database Environment & Connection Architecture

- **DBMS Engine:** MariaDB 11.8.8 / MySQL 8.0 (InnoDB engine, `utf8mb4` character set).
- **Transaction Manager:** `src/db.py` provides an atomic `transaction()` context manager with `autocommit=False` that automatically commits on success and rolls back on any constraint or business violation.
- **Access Control & Admin Login:** A dedicated portal (`/login`) authenticates race administrators against the `admin_user` table. Public visitors access all read operations and public registration (forced to `'Pending'` with no status choice); privileged write operations require an admin session.
- **Security & Sanitization:** 100% of user inputs are passed as parameterized tuples (`%s`), completely neutralizing SQL injection vectors.
- **Data Completeness:** Uses `LEFT JOIN` and `CONCAT_WS` to handle nullable attributes (such as unrecorded official IDs and competitors with single-word names) without data loss.

2 Query (Read) Operations Implementation

2.1 Feature R1: Approved Entries per Edition (Read Operation 1)

- **Route:** `GET /entries` | **Function:** `query_approved_entries(edition_year, category_id)`
- **Description:** Filters confirmed competitors by edition and category. Joins 6 tables using `CONCAT_WS` for participant names.

```
SELECT
    ce.entry_id, ce.bib_number, ce.edition_year, ce.registration_status, ce.registration_time,
    c.category_name, pe.carrier_id,
    CONCAT_WS(' ', p1.first_name, p1.last_name) AS carrier_name, co1.country_name AS
        carrier_country,
    pe.carried_id,
    CONCAT_WS(' ', p2.first_name, p2.last_name) AS carried_name, co2.country_name AS
        carried_country,
    te.team_name
FROM competition_entry ce
JOIN category c ON ce.category_id = c.category_id
LEFT JOIN pair_entry pe ON ce.entry_id = pe.entry_id
LEFT JOIN participant p1 ON pe.carrier_id = p1.participant_id
LEFT JOIN country co1 ON p1.country_code = co1.country_code
LEFT JOIN participant p2 ON pe.carried_id = p2.participant_id
LEFT JOIN country co2 ON p2.country_code = co2.country_code
LEFT JOIN team_entry te ON ce.entry_id = te.entry_id
WHERE ce.edition_year = %s AND ce.registration_status = 'Approved'
  AND (%s IS NULL OR ce.category_id = %s)
ORDER BY ce.bib_number IS NULL, ce.bib_number, ce.entry_id;
```

2.2 Feature R2: Participant Name Search (Read Operation 2)

- **Route:** `GET /search` | **Function:** `query_name_search(name_fragment)`
- **Description:** Wildcard pattern matching (`LIKE %s`) across full names, displaying nationality, editions, categories, and roles.

```
SELECT
    p.participant_id, CONCAT_WS(' ', p.first_name, p.last_name) AS full_name,
    p.first_name, p.last_name, p.approval_status, co.country_name,
    ce.entry_id, ce.edition_year, ce.bib_number, c.category_name,
    pi.participant_role, ce.registration_status
FROM participant p
JOIN country co ON p.country_code = co.country_code
LEFT JOIN participates_in pi ON p.participant_id = pi.participant_id
LEFT JOIN competition_entry ce ON pi.entry_id = ce.entry_id
LEFT JOIN category c ON ce.category_id = c.category_id
WHERE CONCAT_WS(' ', p.first_name, p.last_name) LIKE %s
ORDER BY p.participant_id, ce.edition_year DESC;
```

2.3 Feature R3: Entries per Category Breakdown (Read Operation 3)

- **Route:** `GET /categories` | **Function:** `query_entries_per_category(edition_year)`
- **Description:** Aggregates total, approved, and pending registration volume per category for a given edition using conditional sums.

```
SELECT
    hc.edition_year, c.category_id, c.category_name,
    COUNT(ce.entry_id) AS total_entries,
    COALESCE(SUM(CASE WHEN ce.registration_status = 'Approved' THEN 1 ELSE 0 END), 0) AS
        approved_entries,
    COALESCE(SUM(CASE WHEN ce.registration_status = 'Pending' THEN 1 ELSE 0 END), 0) AS
        pending_entries
FROM has_category hc
JOIN category c ON hc.category_id = c.category_id
LEFT JOIN competition_entry ce
    ON hc.edition_year = ce.edition_year AND hc.category_id = ce.category_id
WHERE (%s IS NULL OR hc.edition_year = %s)
GROUP BY hc.edition_year, c.category_id, c.category_name
ORDER BY hc.edition_year DESC, c.category_id;
```

2.4 Feature R4: Heat Results & Time Differences (Read Operation 4)

- **Route:** `GET /results`
- **Function:** `query_heat_result_sheet(edition_year, race_no, heat_no)` in `src/services.py`
- **Description:** Computes final time (`raw_time + penalty_seconds`) and delta to winner via the window function `MIN(...) OVER ()`. Handles nullable officials with `LEFT JOIN`.

```
SELECT
    rr.edition_year, rr.race_no, rr.heat_no, rr.`rank`, rr.result_status,
    ce.entry_id, ce.bib_number, te.team_name,
    CONCAT_WS(' ', p1.first_name, p1.last_name) AS carrier_name,
    CONCAT_WS(' ', p2.first_name, p2.last_name) AS carried_name,
    rr.raw_time, rr.penalty_seconds,
    CASE WHEN rr.result_status = 'Finished' THEN (rr.raw_time + rr.penalty_seconds) ELSE NULL
        END AS final_time,
    CASE
        WHEN rr.result_status = 'Finished' THEN
            ROUND((rr.raw_time + rr.penalty_seconds) -
                  MIN(CASE WHEN rr.result_status = 'Finished' THEN (rr.raw_time +
                      rr.penalty_seconds) END) OVER (), 3)
        ELSE NULL
    END AS time_diff,
    CONCAT_WS(' ', o.first_name, o.last_name) AS official_name
FROM records_result rr
JOIN competition_entry ce ON rr.entry_id = ce.entry_id
LEFT JOIN pair_entry pe ON ce.entry_id = pe.entry_id
LEFT JOIN participant p1 ON pe.carrier_id = p1.participant_id
LEFT JOIN participant p2 ON pe.carried_id = p2.participant_id
LEFT JOIN team_entry te ON ce.entry_id = te.entry_id
LEFT JOIN official o ON rr.official_id = o.official_id
WHERE rr.edition_year = %s AND rr.race_no = %s AND rr.heat_no = %s
ORDER BY
    CASE WHEN rr.result_status = 'Finished' THEN 0 ELSE 1 END,
    rr.`rank` IS NULL, rr.`rank`, final_time;
```

2.5 Feature R5: Category Average Finish Times (Read Operation 5)

- **Route:** `GET /categories#averages` | **Function:** `query_average_final_times(edition_year)`
- **Description:** Performs statistical aggregation across competition categories for finished entries, computing finisher count, average final time, fastest time, and slowest time.

```
SELECT
    c.category_id, c.category_name,
    COUNT(rr.entry_id) AS total_finishes,
    ROUND(AVG(rr.raw_time + rr.penalty_seconds), 3) AS avg_final_time,
    ROUND(MIN(rr.raw_time + rr.penalty_seconds), 3) AS fastest_final_time,
    ROUND(MAX(rr.raw_time + rr.penalty_seconds), 3) AS slowest_final_time
FROM records_result rr
JOIN competition_entry ce ON rr.entry_id = ce.entry_id
JOIN category c ON ce.category_id = c.category_id
WHERE rr.result_status = 'Finished'
  AND (%s IS NULL OR rr.edition_year = %s)
GROUP BY c.category_id, c.category_name
ORDER BY avg_final_time ASC;
```

3 Update (Write) Operations Implementation

3.1 Feature W1: Register a Pair (Write Operation 1 — Create)

- **Route:** `POST /register` | **Function:** `write_register_pair(...)` in `src/services.py`
- **Description:** Multi-table atomic transaction taking text inputs for carrier and carried competitors. Public entries default to `'Pending'` (no option to choose status); authenticated admins may optionally choose status. Automatically inserts new competitor records into `participant` with `approval_status`.

```
-- Step 1: Validate edition offers chosen category
SELECT 1 FROM has_category WHERE edition_year = %s AND category_id = %s;

-- Step 2: Validate country existence & check if competitor already exists in DB
SELECT country_code FROM country WHERE country_code = %s;
SELECT participant_id, first_name, last_name, date_of_birth, weight_kg, country_code,
    approval_status
FROM participant
WHERE LOWER(first_name) = LOWER(%s)
  AND ((last_name IS NULL AND %s IS NULL) OR LOWER(last_name) = LOWER(%s))
  AND country_code = %s LIMIT 1;

-- Step 3: Insert new competitor into participant table with approval_status
INSERT INTO participant (first_name, last_name, date_of_birth, weight_kg, country_code,
    approval_status)
VALUES (%s, %s, %s, %s, %s, %s);

-- Step 4: Insert master competition entry record
INSERT INTO competition_entry (edition_year, category_id, bib_number, registration_status)
VALUES (%s, %s, %s, %s);

-- Step 5: Insert pair_entry subtype specialization using generated entry_id
INSERT INTO pair_entry (entry_id, carrier_id, carried_id)
VALUES (%s, %s, %s);

-- Step 6: Insert Carrier and Carried roles into participates_in
INSERT INTO participates_in (entry_id, participant_id, participant_role)
VALUES (%s, %s, 'Carrier'), (%s, %s, 'Carried');

-- Step 7: Record participant-edition history in participates_in_edition
INSERT IGNORE INTO participates_in_edition (participant_id, edition_year)
VALUES (%s, %s), (%s, %s);
```

3.2 Feature W2: Change Participant Country (Write Operation 2 — Update)

- **Route:** `POST /participants/update-country`
- **Function:** `write_update_participant_country(participant_id, new_code)` in `src/services.py`
- **Description:** Admin-only write operation (protected by `@admin_required`). Updates participant nationality with immediate foreign key verification against the reference `country` table.

```
-- Step 1: Ensure destination country code exists in country table
SELECT country_name FROM country WHERE country_code = %s;

-- Step 2: Execute atomic parameterized update
UPDATE participant
SET country_code = %s
WHERE participant_id = %s;
```

3.3 Feature W3: Delete Pending Entry (Write Operation 3 — Delete) & Approval

- **Routes:** `POST /entries/delete/<entry_id>` | `POST /entries/approve/<entry_id>`
- **Functions:** `write_delete_pending_entry(entry_id)`
  `write_approve_pending_entry(entry_id)`
- **Description:** Admin-only write operation (protected by `@admin_required`). Allows administrators to approve pending entries or safely delete them with edition cleanup and automatic orphan competitor removal.

```
-- Step 1: Verify entry exists & fetch edition_year / participants
SELECT entry_id, edition_year, registration_status FROM competition_entry WHERE entry_id = %s;
SELECT participant_id FROM participates_in WHERE entry_id = %s;

-- Step 2: Reject deletion if heat results already exist; otherwise cascade clean-up
SELECT 1 FROM records_result WHERE entry_id = %s;
DELETE FROM participates_in WHERE entry_id = %s;
DELETE FROM pair_entry WHERE entry_id = %s;
DELETE FROM team_entry WHERE entry_id = %s;
DELETE FROM assigned_to WHERE entry_id = %s;
DELETE FROM competition_entry WHERE entry_id = %s;

-- Step 3: Remove edition participation & orphaned competitors with no remaining entries
DELETE FROM participates_in_edition WHERE participant_id = %s AND edition_year = %s;
DELETE FROM participant_contact WHERE participant_id = %s;
DELETE FROM participant WHERE participant_id = %s;

-- [Approval Workflow]: Confirm entry & promote participant approval_status
UPDATE competition_entry SET registration_status = 'Approved' WHERE entry_id = %s;
UPDATE participant SET approval_status = 'Approved'
WHERE participant_id IN (SELECT participant_id FROM participates_in WHERE entry_id = %s);
```

4 Database Integrity & Security Architecture

1. **ACID Transaction Management:** Multi-table write workflows (W1 and W3) utilize the Python context manager `db.transaction()` with `autocommit=False`. If any constraint, foreign key, or business check fails, an automatic rollback is immediately triggered.
2. **Protection Against SQL Injection:** Every SQL statement executed across the Flask backend strictly uses parameter substitution with `%s` placeholders. No manual string concatenation is used for user inputs.
3. **Database-Level Disjoint Triggers:** The database schema enforces disjoint specialization natively via triggers (`trg_pair_not_team` and `trg_team_not_pair`), ensuring no entry can simultaneously exist in `pair_entry` and `team_entry`.
4. **Safe Handling of Nullable Columns:** Left outer joins (`LEFT JOIN`) and `CONCAT_WS` ensure that unrecorded attributes (e.g., unpublished officials or participants with single-word names) are handled gracefully without omitting rows or breaking page rendering.
5. **Participant Lifecycle & Role-Based Access Control:** Competitor text inputs dynamically set `approval_status` (`'Pending'` or `'Approved'`). Deleting a pending entry purges `participates_in_edition` and deletes orphaned competitors. Race administrators authenticate via parameterized SQL against `admin_user` with secure password hashing; public writes are strictly limited to pair registration (forced to `'Pending'`).

5 Verification & Test Execution

All 8 features and underlying queries have been verified against the live MariaDB 11.8 database populated with the official Eukonkanto championship dataset:

- **Read Queries:** R1 returns 82 approved entries for 2026; R2 retrieves complete participation history and approval status; R3 aggregates counts across 5 categories; R4 computes heat results and time deltas across 8 heats; R5 computes statistical averages per category.
- **Write Queries:** W1 inserts new competitors via text input with initial `approval_status`, creates `competition_entry` and `pair_entry`, and records 2027 participation; W2 updates participant nationality; W3 cascades deletion of pending entries, cleans up edition participation, deletes orphaned competitors from `participant`, and synchronizes approval status upon approval.